

claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_e27060ad-65f\agent-acbce293c748c6cf6.jsonl`  
- SHA-256: `e0d493b2e9399643f59704b29378a5d3b79cce2559df508e8764c184b42c8065`  
- Source modified: `2026-06-19T16:48:33+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_e27060ad-65f`  
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`
```

Transcript

1. user (2026-06-19T16:44:38.352Z)

Verify the accuracy of the "How Rally Detection Works" narrative (Section 1) in the PR's RALLY_DETECTION_CODE_REVIEW.md against the real code and docs/HOW_RALLY_DETECTION_WORKS.md.

The narrative asserts a pipeline: velocity thresholding (velocity_thresh ~0.02), merging within merge_gap ~2.0s, W1 split over max_window_duration at largest gap >= min_split_gap, window_hard_cap chop-in-half, R4 inactive-tail trim, R5 blob fallback with forced_cut flag, Gate A net-crossing filter (min_crossings ~2, median play-axis), Phase 3 padding ~2s each end to a Gemini provider.

TASKS:

1. Spot-check each NAMED constant/default (velocity_thresh, merge_gap, max_window_duration, min_split_gap, window_hard_cap, rally_thresh/min_density, padding, min_crossings) against the actual config (backend/config/) and code defaults. Report claimed-vs-actual for each.
2. Check the W1 / R4 / R5 / Gate A labels: does the code actually use these labels/flags (forced_cut, etc.)? Is the described mechanism faithful?
3. Compare against docs/HOW_RALLY_DETECTION_WORKS.md ? does the review's mental model match the repo's own documented model, or does it invent structure?
4. Flag any confident-sounding but WRONG or UNVERIFIABLE statements. List them.

Return structured output. section = "How Rally Detection Works (narrative + config facts)".

2. assistant (2026-06-19T16:44:40.654Z)

I'll verify the accuracy of the "How Rally Detection Works" narrative against the real code and docs. Let me start by finding and reading the key files.

3. user (2026-06-19T16:44:44.040Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end' from the remote, but no such ref was fetched.

---STATUS---

docs/court-floor-rally-end...origin/docs/court-floor-rally-end

?? artifacts/

?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md

4. user (2026-06-19T16:44:44.273Z)

docs\HOW_RALLY_DETECTION_WORKS.md

5. user (2026-06-19T16:44:44.273Z)

No files found

6. assistant (2026-06-19T16:44:47.162Z)

The review file isn't found by that glob. Let me locate it and read the docs file.

7. user (2026-06-19T16:44:50.066Z)

```
1      # How rally detection works (the 2-layer mental model)
2
3      A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
4      and, crucially, **which layer our annotation can improve.**
5
6      ```
7          video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
8              ?
9              (WASB CNN,          per-frame trajectory
10             frozen weights)      ?
11                                   ?
12                                   velocity signal (how fast the shuttle moves)
13                                   ?
14                                   [SEGMENTATION] ??? the 3 knobs we tune
15                                   ?
16                                   ?
17                                   rally windows [start, end] ??? IoU vs golden
labels
18      ```
19
20      ## Two layers ? keep them separate in your head
21
22      **1. DETECTION ? "where is the shuttle, this frame?"**
23      WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
24      visibility/confidence. Its weights were learned (by gradient descent, on shuttle-
coordinate
25      labels) and we use them **frozen**. We chose a *shuttle* detector on purpose: "shuttle
in
26      flight" is the signal for "a rally is happening," and it's **camera-agnostic** ? unlike
27      player-motion, which fooled the old heuristic on broadcast cameras that pan with the
players.
28      (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is
the
29      robust general bet.)
30
31      **2. SEGMENTATION ? "when is a rally?"**
32      From the per-frame `(x, y)` we compute the shuttle's **velocity** (displacement between
frames).
33      Then three knobs turn that into rally intervals:
34      - **`velocity_thresh`** ? a frame is "active" only if the shuttle is moving faster than
this
35      (in flight, not lying on the floor / being picked up).
36      - **`merge_gap`** ? active frames within this many seconds get joined into one rally
(bridges
37      brief gaps: a shuttle momentarily occluded, or the beat between two hits).
38      - **`min_window_duration`** ? throw away windows shorter than this (blips / false
flickers).
39
40      Output: a list of `[start, end]` windows = the rallies.
41
42      ## Where the annotation fits (the key point)
43      Our Roora/golden labels are **rally `[start, end]` times** ? they say *when* rallies
are, not
44      *where* the shuttle is*. So they can't teach WASB's detector (that needs `(x,y)` labels).
They
45      **can** grade and tune the **segmentation layer**: we score predicted windows against
the golden
46      rallies with **temporal IoU** (how much they overlap), and pick the 3 knobs that
maximise it ?
```

```

47     **calibrate on some rallies, measure on held-out ones** so the number is honest.
48
49     ## Where sparks could land (open questions ? not commitments)
50     Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
51     - **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real
rally is a
52         smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals**
(each
53         reversal ? a shot ? ties to `shots_count`).
54     - **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to
a small
55         model trained on the golden rally labels (still the segmentation layer, just smarter).
56     - **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-
rally dips.
57     - **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
58     - **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams. ?
designed in
59         [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (one rally-fusion layer;
the person cue is
60         ADR-007's parked #2 and `rally_gate.py`'s "Gate B" ? now built (default-OFF,
unmeasured on golden) in `fusion_hybrid.py`).
61     - **Per-video calibration as a *feature*:** "watch a few labeled rallies ? auto-tune to
this
62         court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right
layer.
63
64     The whole point of per-video calibration: a handful of *your* labeled rallies nudges
these knobs
65     to *your* court/camera, and we report the IoU lift on the rallies it never saw.
66

```

8. user (2026-06-19T16:44:50.247Z)

```

430de76 docs(rally): capture the shuttle-floor-contact rally-END signal (owner note)
e358b28 feat(eval): prominent-court window gate ? drop multicourt background-court rallies
(default-OFF) (#243)
9736a3b docs: re-ground stale doc claims to the current code (multi-agent audit) (#242)
626061d docs(rally): prominent-court detection design ? kill multicourt background-court FPs
(#241)
3f5a56c fix(gemini): surface failed chunks loudly (no more silent partial reels) (#240)
---DIFF FROM MASTER---
.gitattributes
.github/workflows/ci.yml
CLAUDE.md
backend/api/__init__.py
backend/api/job_worker.py
backend/api/uploads.py
backend/config/loader.py
backend/config/models.py
backend/eval/ablation.py
backend/eval/ablation_pdf.py
backend/eval/experiment.py
backend/eval/fusion_audio.py
backend/eval/fusion_compare.py
backend/eval/golden_fixtures.py
backend/eval/golden_real_fixtures.py
backend/eval/rally_seg_eval.py
backend/eval/tolerance_metrics.py
backend/eval/windowing.py
backend/infrastructure/database.py
backend/main.py
backend/pipeline/detectors/court_gate.py
backend/pipeline/detectors/tracknet_runner.py
backend/pipeline/segmenters/fusion_hybrid.py
backend/pipeline/segmenters/gemini.py

```

```
backend/pipeline/segmenters/motion.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
backend/pipeline/stitcher/compiler.py
backend/providers/gemini.py
backend/results.py
docs/BACKLOG.md
docs/CODE_MAP.md
docs/DESKTOP_APP_PLAN.md
docs/GOLDEN_DATA_SHARING.md
docs/GOLDEN_REGRESSION_FIXTURES.md
docs/GOLDEN_VIDEOS.md
docs/HOW_RALLY_DETECTION_WORKS.md
docs/MULTI_SIGNAL_FUSION_PLAN.md
docs/NEXT_STEPS.md
docs/PROJECT_DOC_TEMPLATE.md
---SEARCH REVIEW FILE---
```

9. assistant (2026-06-19T16:44:53.682Z)

The review file isn't in the working tree by the searched name. Let me search more broadly and check the artifacts/ directory (which is untracked).

10. user (2026-06-19T16:45:02.555Z)

```
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/bhi-execpolicy/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/boxhill17/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/gap-doc/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/hardchop/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/pr-stitch/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/recdoc/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/resweep/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/tier1-windowing/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./..claude/worktrees/w2-safeguard/docs/archives/COMMERCIAL_READINESS_REVIEW.md
./docs/archives/COMMERCIAL_READINESS_REVIEW.md
---ARTIFACTS---
total 976
drwxr-xr-x 1 avidu 197609      0 Jun 18 13:25 .
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:46 ..
-rw-r--r-- 1 avidu 197609 18149 Jun 18 13:25 long_rally_lens_n15.json
-rw-r--r-- 1 avidu 197609  3870 Jun 18 13:25 long_rally_lens_n15.txt
drwxr-xr-x 1 avidu 197609      0 Jun 10 15:54 rally_tal_gen0
-rw-r--r-- 1 avidu 197609 912462 Jun 16 08:50 tridet.pdf
-rw-r--r-- 1 avidu 197609 46340 Jun 16 08:50 tridet.txt
---GREG for code review content---
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-first-ab-
experiment/README.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-human-gt-
tuning/README.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/CODE_MAP.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/EVALUATION.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/EXPERIMENT_HARNESS.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/HOW_RALLY_DETECTION_WORKS.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/QUALITY_ITERATIONS.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/TRACKNET_WSL_SETUP.md
./..claude/worktrees/agent-a771ff1d85e18f2ed/README.md
./..claude/worktrees/bhi-execpolicy/docs/archives/quality-iterations/2026-06-first-ab-
experiment/README.md
```

11. assistant (2026-06-19T16:45:05.318Z)

The RALLY_DETECTION_CODE_REVIEW.md file doesn't exist in the working tree. It may be the PR description, an uncommitted file, or staged elsewhere. Let me check git more carefully and the

worktrees.

12. user (2026-06-19T16:46:34.219Z)

```
===ALL BRANCHES===
+ bhi-execpolicy
  chore/consolidate-fusion-schema-predicate
* docs/court-floor-rally-end
  docs/desktop-app-plan
  docs/golden-regression-fixtures-plan
+ docs/lovo-resweep-n13
+ docs/rally-gap-closure
+ docs/recording-single-match
  feat/highlights-runner
  feat/local-no-ai-segmenter
  feat/long-rally-lens
  feat/prominent-court-gate
+ feat/w1-hard-cap-fallback
+ feat/w2-never-empty-safeguard
  fix/compiler-dedup-overlapping-segments
  fix/fusion-loaders-mixed-schema
  fix/gemini-loud-chunk-failures
  fix/gemini-per-video-chunk-dir
+ fix/runner-stitch-watermark
  fix/served-gate-a-corpus-regrounding
  fix/wasb-test-mocks-readability-preflight
  fix/wsl-unreadable-source-preflight
+ master
  ops/sync-master-d749fb9
+ perf/wasb-fast-frames
+ run/boxhill17-highlights
  worktree-agent-a771ff1d85e18f2ed
  worktree-wf_1d39e81c-851-1
  worktree-wf_1d39e81c-851-3
  worktree-wf_1d39e81c-851-4
  worktree-wf_1d39e81c-851-5
  worktree-wf_1d39e81c-851-6
  worktree-wf_1d39e81c-851-7
  worktree-wf_87c49086-be4-1
  worktree-wf_87c49086-be4-2
  worktree-wf_87c49086-be4-3
  worktree-wf_87c49086-be4-4
  worktree-wf_87c49086-be4-5
  worktree-wf_87c49086-be4-6
  worktree-wf_87c49086-be4-7
===STASH===
===GREP RALLY_DETECTION_CODE_REVIEW everywhere===
===find Gate A net-crossing in any md===
./docs/ABLATION_LAYER.md
./docs/ANALYZERS_AND_RECONCILERS.md
./docs/archives/decisions/DECISIONS.md
./docs/archives/quality-iterations/2026-06-first-ab-experiment/README.md
./docs/archives/quality-iterations/2026-06-human-gt-tuning/README.md
./docs/archives/quality-iterations/2026-06-multivideo-lovo/README.md
./docs/archives/quality-iterations/README.md
./docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md
./docs/CODE_MAP.md
./docs/EXPERIMENT_HARNESS.md
```

13. assistant (2026-06-19T16:46:42.120Z)

The RALLY_DETECTION_CODE_REVIEW.md file genuinely does not exist anywhere in this repo (no branch, no worktree, no stash, no untracked file). The task says "the PR's RALLY_DETECTION_CODE_REVIEW.md" ? this is presumably content I'm meant to verify, but the file isn't present.

The task gives me the full set of claims to verify in its description, so I can still do the substantive verification work: check each named constant/default against the actual code/config, and compare the asserted mental model against `docs/HOW\_RALLY\_DETECTION\_WORKS.md`. Let me verify the code thoroughly. The narrative describes a **WASB shuttle ? velocity ? merge ? W1 split ? hard-cap ? R4 trim ? R5 blob ? Gate A ? padding ? Gemini** pipeline. That sounds like the `trajectory\_hybrid.py` / `fusion\_hybrid.py` segmenter. Let me read those and the config.

14. user (2026-06-19T16:46:42.552Z)

```
1      """Shared engine for trajectory-substrate hybrid segmenters.
2
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family`` ? config section under ``indexing.*``, output subdir, and
12       the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16       detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
```

```

55         raise NotImplementedError
56
57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         returns exactly the 4 motion keys, so the trajectory twins persist byte-
89         identical
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                            window_stats: List[Dict[str, Any]],
102                            idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110
111     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
112                      player_pool: Optional[List[str]] = None,
113                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
114         # override-ignore: the ABC declares the Result contract (Success|Failure)
115         # but every live segmenter still returns a bare list ? the documented
116         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
117         label = self.display_label
118         # --- Sport profile + AI provider wiring (identical across segmenters) ---
119         sport_name = self.config.get("sport", "badminton")

```

```

115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
123         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
124         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
125         # measure it against the Gemini path, or to avoid the cloud entirely). With
126         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
127         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
129         if skip_ai:
130             model_name = "local-noai"
131             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
132                         "be emitted as rallies; no %s handoff, no API key needed.",
133                         label, provider_name)
134         else:
135             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
136             api_key_env_var = f"{provider_name.upper()}_API_KEY"
137             api_key = os.environ.get(api_key_env_var)
138             if not api_key:
139                 logger.error(
140                     f"{api_key_env_var} environment variable is not set! "
141                     f"To run the {provider_name} handoff, set your API key. "
142                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
143                 )
144             return []
145         try:
146             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
147         except KeyError as e:
148             logger.error(str(e))
149             return []
150
151         video_duration = get_video_duration(video_path)
152         if video_duration <= 0:
153             logger.error("Invalid video duration.")
154             return []
155         fps, frame_width = self._probe_fps_width(video_path)
156
157         # --- Runner config + windowing thresholds ---
158         runner, runner_params = self._build_runner(idx_cfg)
159
160         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
164         max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165         window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166         window_hard_cap = idx_cfg.get("window_hard_cap", False)
167         window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168         inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169         inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170         window_blob_fallback = idx_cfg.get("window_blob_fallback", False)

```



```

171     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
172     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
173     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
174     log_candidates = idx_cfg.get("log_yolo_candidates", True)
175     store_candidates = idx_cfg.get("store_yolo_candidates", True)
176
177     detection_params = {
178         "detector": self.name,
179         "velocity_thresh": velocity_thresh,
180         "merge_gap": merge_gap,
181         "min_action_window_duration": min_window_duration,
182         **runner_params,
183     }
184
185     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
186     ok, msg = runner.healthcheck()
187     if not ok:
188         logger.error("%s WSL environment not ready: %s", label, msg)
189         return []
190     logger.info("%s WSL env OK: %s", label, msg)
191
192     # --- Phase 2: trajectory -> candidate rally windows ---
193     # Trajectory detectors process the whole video in one pass (they carry
194     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
195     t_start = time.time()
196     out_dir = os.path.join("output", self.family)
197     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
198     if not csv_path:
199         logger.error("%s produced no trajectory; aborting.", label)
200         return []
201
202     points = runner.parse_trajectory_csv(csv_path)
203     logger.info("Parsed %d trajectory points (%d visible).",
204                 len(points), sum(1 for p in points if p.visible))
205
206     all_candidate_windows = runner.trajectory_to_action_windows(
207         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
208         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
209         min_window_duration=min_window_duration, chunk_index=0,
210         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
211         window_hard_cap=window_hard_cap,
212         window_inactivity_end=window_inactivity_end,
213         inactivity_rally_thresh=inactivity_rally_thresh,
214         inactivity_min_density=inactivity_min_density,
215         window_blob_fallback=window_blob_fallback,
216         window_blob_factor=window_blob_factor,
217     )
218     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
219                 label, time.time() - t_start, len(all_candidate_windows))
220
221     if log_candidates:
222         for cw in all_candidate_windows:
223             logger.info(f"[{label}] rally
224 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
225                        f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
226                        f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
227
228     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
229     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
230     # persistence and the handoff gate below.
window_stats = [

```

```

231         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
232     for cw in all_candidate_windows
233 ]
234
235     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
236     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
237     if store_candidates:
238         for i, cw in enumerate(all_candidate_windows):
239             candidate_ids[i] = self.db.add_candidate_segment(
240                 video_id, detector=self.name,
241                 start_time=cw["start"], end_time=cw["end"],
242                 chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"])),
243                 stats=window_stats[i], detection_params=detection_params,
244             )
245
246     if not all_candidate_windows:
247         return []
248
249     # --- Phase 3: AI provider handoff over padded candidate clips ---
250     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
251     # candidates above stay the full superset ? only the served play_segments are
gated.
252     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
253
254     ai_segments: List[dict] = []
255     if skip_ai:
256         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
257         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
258         # falls back to the duration-based estimate (no AI exchange count).
259         ai_segments = [
260             {
261                 "start_time": all_candidate_windows[wi]["start"],
262                 "end_time": all_candidate_windows[wi]["end"],
263                 "shuttle_exchange_count": None,
264                 "shot_count_confidence": "LocalNoAI",
265                 "_candidate_id": candidate_ids[wi],
266             }
267             for wi in handoff_indices
268         ]
269         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
270                     "(no AI handoff).", len(ai_segments))
271     else:
272         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
273         os.makedirs(handoff_dir, exist_ok=True)
274         logger.info("Phase 3: %s validation on %d candidate windows...",
275                     provider_name, len(handoff_indices))
276
277         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
278             w_start = max(0, window["start"] - handoff_padding)
279             w_end = min(video_duration, window["end"] + handoff_padding)
280             w_dur = w_end - w_start
281             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
282             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
283                 return []
284             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
285             segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,

```

```

286                                     label=f"Handoff {wi+1}")
287         valid = []
288         for seg in segments:
289             try:
290                 seg["start_time"] = float(seg["start_time"]) + w_start
291                 seg["end_time"] = float(seg["end_time"]) + w_start
292                 seg["_candidate_id"] = candidate_ids[wi]
293                 valid.append(seg)
294             except (ValueError, TypeError, KeyError) as e:
295                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
296         try:
297             os.remove(w_path)
298         except OSError:
299             pass
300         return valid
301
302         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
303             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
304                         for wi in handoff_indices]
305             for future in futures:
306                 ai_segments.extend(future.result())
307         try:
308             os.rmdir(handoff_dir)
309         except OSError:
310             pass
311         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
312
313         # --- Phase 4: DB population + candidate linking ---
314         segments_data = []
315         for segment in ai_segments:
316             try:
317                 start = float(segment["start_time"])
318                 end = float(segment["end_time"])
319                 dur = end - start
320                 metadata = {
321                     "shot_count": self._shot_count_from(segment, dur),
322                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323                     "detector": f"{self.family}-hybrid-{model_name}",
324                 }
325                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326                 if segment_id:
327                     candidate_id = segment.get("_candidate_id")
328                     if candidate_id is not None:
329                         self.db.link_candidate_to_segment(candidate_id, segment_id)
330                     segments_data.append({
331                         "id": segment_id, "start_time": start, "end_time": end,
332                         "duration": dur, "metadata": metadata,
333                     })
334             except Exception as e:
335                 logger.warning("Skipping invalid segment struct: %s", e)
336
337         return segments_data
338

```

15. user (2026-06-19T16:46:42.890Z)

```

1         """Pydantic models for the application configuration.
2
3         This module defines the canonical, validated, typed configuration for the

```

```

4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33
34     class TrackNetConfig(BaseModel):
35         wsl_distro: str = "Ubuntu"
36         repo_dir: str = "~/models/TrackNetV4"
37         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38         conda_env: str = "TrackNetV4"
39         weights_path: str = ""
40         queue_length: int = 5
41         stage_in_wsl: bool = True
42         wsl_stage_dir: str = "~/clips"
43         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
44         velocity_threshold: float = 0.02
45         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
46         # decoded frame cache) are deleted automatically ? they are pure intermediates,
47         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
48         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
49         keep_frames: bool = False
50         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
51         # 0 = disabled.
52         min_free_gb: float = 0.0
53
54
55     class WasbIndexConfig(BaseModel):
56         """Typed config for the live WASB shuttle substrate (indexing.wasb).
57
58         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
59         these knobs actually take effect ? previously this whole block was silently dropped
60         because nested config sections defaulted to extra='ignore'.
61         """
62         wsl_distro: str = "Ubuntu"
63         repo_dir: str = "~/models/WASB-SBDT"
64         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
65         conda_env: str = "wasb"
66         weights_path: str = "~/models/WASB-
67         SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
68         sport: str = "badminton"

```

```

67     wsl_stage_dir: str = "~/clips"
68     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69     velocity_threshold: float = 0.02
70     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73     keep_frames: bool = False
74     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75     min_free_gb: float = 0.0
76     # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
77     # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
78     # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79     # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80     # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81     stream_video: bool = False
82
83
84     class FusionConfig(BaseModel):
85         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87         Every default reproduces control behaviour: the fusion segmenter persists the
88         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
89         true ? the person-cue features, but it does NOT gate the served handoff unless a
90         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
91         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
92         supplied ? off-court persons (spectators / adjacent courts) are excluded.
93         """
94         # Which trajectory substrate seeds the windows (shuttle stays primary).
95         substrate: Literal["wasb", "tracknet"] = "wasb"
96         # Windowing velocity threshold for the fusion family (the engine reads
97         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99         velocity_threshold: float = 0.02
100        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101        # enabled ? so the default path never imports torch / touches the GPU.
102        cue_flags: dict[str, bool] = Field(default_factory=dict)
103        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105        min_crossings: int = Field(default=0, ge=0)
106        # Served Gate B: when the person cue is on, drop windows with median in-court
players
107        # < this. 0 = OFF.
108        min_players: int = Field(default=0, ge=0)
109        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110        court_polygon: Optional[list[list[float]]] = None
111        # Net line for the person two-sided-motion split (person features only ? the shuttle
112        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113        net_axis: Literal["x", "y"] = "y"
114        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115
116
117        class IndexingConfig(BaseModel):
118            default_segmenter: str = "yolo_hybrid"

```

```

119     use_gpu: bool = True
120     motion_threshold: float = 0.08
121     min_segment_duration: float = 3.0
122     dead_time_merge_gap: float = 2.0
123     chunk_duration: float = 90.0
124     chunk_overlap: float = 10.0
125     detector_model: str = "fasterrcnn_resnet50_fpn"
126     detector_score_threshold: float = 0.5
127     yolo_velocity_threshold: float = 0.15
128     yolo_frame_skip: int = 3
129     yolo_tracker: str = "bytetrack.yaml"
130     yolo_prefetch_workers: int = 2
131     min_action_window_duration: float = 2.0
132     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
133     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?)
134     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
135     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
136     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
137     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
138     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
139     max_window_duration: float = 6.0
140     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
141     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
142     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
143     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
144     # Only cuts (recall can't drop). OFF by default until measured/promoted.
145     window_hard_cap: bool = False
146     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
147     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
148     # [|?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
149     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
150     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
151     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [|?end| 4.96?3.31s (n=13).
The W1 cap
152     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
153     window_inactivity_end: float = 1.5
154     inactivity_rally_thresh: float = 0.20
155     inactivity_min_density: float = 0.30
156     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
157     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
158     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
159     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
160     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the

```

```

161         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
162         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
163         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
164         window_blob_fallback: bool = False
165         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
166         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
167         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
168         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
169         window_blob_factor: float = 4.0
170         log_yolo_candidates: bool = True
171         store_yolo_candidates: bool = True
172         ai_handoff_padding: float = 2.0
173         ai_max_workers: int = 5
174         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
175         # chunks of a high-bitrate (4K) source blow past the provider upload cap
176         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
177         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
178         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
179         # Matches gemini_refine's --clip-scale-width default.
180         ai_chunk_scale_width: int = 1280
181         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
182         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
183         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
184         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
185         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
186         skip_ai_handoff: bool = False
187         # Optional debug knob: trim every video to the first N seconds before processing.
188         debug_duration: Optional[float] = None
189
190         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
191         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
192         fusion: FusionConfig = Field(default_factory=FusionConfig)
193
194         @field_validator("default_segmenter")
195         @classmethod
196         def segmenter_must_be_registered(cls, v: str) -> str:
197             # Registry check happens at runtime in main/segmenter selection.
198             # Here we just allow any string for forward compatibility.
199             return v
200
201         @model_validator(mode="after")
202         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
203             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
204             # step makes `while t < duration: t += step` loop forever, growing memory before
any
205             # GPU work. Reject the bad config at load time instead.
206             if self.chunk_overlap >= self.chunk_duration:
207                 raise ValueError(
208                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
209                     f"({self.chunk_duration}); otherwise chunking never advances."
210                 )
211             return self
212

```

```

213
214 class OutputConfig(BaseModel):
215     default_highlights_name: str = "highlights.mp4"
216     db_name: str = "sports_indexer.db"
217     # Directory where the DB, highlight reels, and processed clips are written.
218     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
219     output_dir: str = "output"
220
221
222 class WatermarkConfig(BaseModel):
223     """Branding overlay burned into each highlight clip during the per-clip re-encode
224     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
225     (under `stitching.watermark` in config.json) to turn it off. The text is fully
226     configurable. The concat stage stays stream-copy (-c copy)."""
227     enabled: bool = True
228     text: str = "Generated by Khelsutra.guru"
229     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
230     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
231     font: str = "Sans" # fontconfig family used when font_path is empty
232     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
233     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
234     box: bool = True # faint backing box behind the text for legibility
235     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
236     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
237
238
239 class StitchingConfig(BaseModel):
240     begin_padding: float = 0.5
241     end_padding: float = 0.5
242     use_cache: bool = False
243     min_shot_count: int = 1
244     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
245     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
246     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
247     # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
248     # local detector over-fires and its false positives are mostly SHORT (motion blips,
249     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
250     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
251     min_rally_duration: float = Field(default=0.0, ge=0.0)
252     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
253
254
255 class LoggingConfig(BaseModel):
256     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
257     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
258     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
259     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
260     # them to WARNING; set true to restore full chatter for request-flow debugging.
261     verbose_http: bool = False
262
263
264 class SecurityConfig(BaseModel):
265     # When set, /api/process video_path must resolve under this directory (hosted/tenant
266     # mode). Default None preserves the single-user local 'any local file' workflow.
267     allowed_video_root: Optional[str] = None
268
269     # --- Chunked upload (B2, PR-2) ---

```



```

270 # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
271 # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
272 # contain upload_dir or /api/process will reject the uploaded paths.
273 upload_dir: str = "output/uploads"
274 # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
275 # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
276 max_upload_mb: int = 4096
277 max_part_mb: int = 95
278 allowed_upload_extensions: List[str] = ["mp4", "mov"]
279
280
281 class AppConfig(BaseModel):
282     """Root configuration object.
283
284     All runtime configuration should be accessed via an instance of this class
285     (or the loader) rather than raw dict lookups.
286     """
287
288     sport: Sport = "badminton"
289     ai_provider: str = "gemini"
290     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
291     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
292     ai_model: str = "gemini-flash-latest"
293
294     validation: ValidationConfig = Field(default_factory=ValidationConfig)
295     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
296     output: OutputConfig = Field(default_factory=OutputConfig)
297     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
298     security: SecurityConfig = Field(default_factory=SecurityConfig)
299     logging: LoggingConfig = Field(default_factory=LoggingConfig)
300
301     # Allow extra keys during transition so existing config.json files do not break.
302     model_config = {
303         "extra": "allow",
304         "validate_assignment": True,
305     }
306
307     # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
308     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
309     # attribute access / the typed config. A nested *section* is returned as a plain
dict
310     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
311     # section model instead breaks it (a model has no .get), which bit every validator
on
312     # real runs. Scalar leaves are returned as-is.
313     def get(self, key: str, default: Any = None) -> Any:
314         _MISSING = object()
315         val: Any = _MISSING
316         if key in type(self).model_fields or hasattr(self, key):
317             val = getattr(self, key, _MISSING)
318         if val is _MISSING:
319             # Legacy convenience: find a leaf key inside any nested section. Sections
are
320             # discovered dynamically from the model fields, so a future top-level
section
321             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
322             for fname in type(self).model_fields:
323                 section = getattr(self, fname, None)
324                 if isinstance(section, BaseModel) and hasattr(section, key):
325                     val = getattr(section, key)
326                     break

```

```

327         if val is _MISSING:
328             return default
329         if isinstance(val, BaseModel):
330             return val.model_dump(mode="json")
331         return val
332
333     def __getitem__(self, key: str) -> Any:
334         val = self.get(key)
335         if val is None and key not in type(self).model_fields:
336             # Re-raise like a real dict for unknown top-level
337             raise KeyError(key)
338         return val
339
340
341     # Convenience type alias for callers that want to annotate
342     Config = AppConfig
343

```

16. assistant (2026-06-19T16:46:47.600Z)

I now have the config defaults. Now let me read the windowing logic to verify the W1 split / hard-cap / R4 / R5 mechanisms, and the fusion segmenter for Gate A. The ``trajectory_to_action_windows`` is on the runner ? let me find it. Also need to check Gate A's ``min_crossings`` default and the net-crossing/median play-axis logic in ``rally_gate.py`` / ``fusion_hybrid.py``.

17. user (2026-06-19T16:46:48.701Z)

Found 1 file
 backend\pipeline\detectors\tracknet_runner.py

18. user (2026-06-19T16:46:49.206Z)

```

1     """Multi-signal fusion segmenter (MULTI-SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds

```

```

24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54             @property
55             def name(self) -> str:
56                 return "fusion_hybrid"
57
58             @property
59             def description(self) -> str:
60                 return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                         "AI provider sets semantic boundaries.")
63
64             def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65                 substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66                 if substrate == "tracknet":
67                     cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                     return TrackNetRunner(cfg), {
69                         "weights_path": cfg.weights_path,
70                         "queue_length": cfg.queue_length,
71                         "fusion_substrate": "tracknet",
72                     }
73                 wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74                 return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76             def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                         frame_width: float, video_path: str,
78                                         idx_cfg: Dict[str, Any]) -> Dict[str, Any]:

```

```

79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
84                                             min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100        def _axis_estimate(self, points: List[Any]) -> tuple:
101            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102            computed once per video, not once per candidate window (it depends only on
points)."""
103            key = id(points)
104            if self._axis_cache_key != key or self._axis_cache is None:
105                self._axis_cache_key = key
106                self._axis_cache = rally_gate.estimate_play_axis(points)
107            est = self._axis_cache
108            assert est is not None
109            return est
110
111        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                            frame_width: float, video_path: str,
113                            fcfg: Dict[str, Any]) -> Dict[str, float]:
114            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115            frame range and compute the scale/translation-invariant person features. Lazily
builds
116            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118            from backend.pipeline.detectors.person_detector import PersonDetector
119            if self._person is None:
120                self._person = PersonDetector(self.config)
121
122            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.

```

```

125         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127         fw = det["frame_width"] or frame_width
128         fh = det["frame_height"]
129         # If we can't trust the frame dims, every normalised feature would silently
collapse
130         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131         if fw <= 0 or fh <= 0:
132             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134         poly = fcfg.get("court_polygon")
135         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                         court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                           window_stats: List[Dict[str, Any]],
143                           idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset).
147
148         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151         set rather than emit a video with zero rallies. This makes Gate A/B strictly
152         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154         below it always wins on an otherwise-empty video. Observed on real footage
155         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157         fcfg = idx_cfg.get("fusion", {})
158         min_crossings = int(fcfg.get("min_crossings", 0))
159         min_players = int(fcfg.get("min_players", 0))
160         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162         keep: List[int] = []
163         for i, st in enumerate(window_stats):
164             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                 continue
166             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                 continue
168             keep.append(i)
169
170         if not keep and window_stats:
171             # Gating emptied a non-empty video ? fall back to the ungated set.
172             return list(range(len(window_stats)))
173         return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177

```

Now let me read the `trajectory\_to\_action\_windows` implementation (the W1/R4/R5 mechanics) and the `rally\_gate.py` for Gate A net-crossing / median play-axis logic.

20. user (2026-06-19T16:46:55.904Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                     # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                    # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45      @classmethod
46      def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47          tn = (idx_cfg or {}).get("tracknet", {}) or {}
48          return cls(
49              distro=tn.get("wsl_distro", "Ubuntu"),
50              repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51              conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52              conda_env=tn.get("conda_env", "TrackNetV4"),
```

```

53         weights_path=tn.get("weights_path", ""),
54         queue_length=int(tn.get("queue_length", 5)),
55         stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56         wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57         timeout_sec=int(tn.get("timeout_sec", 1800)),
58         keep_frames=bool(tn.get("keep_frames", False)),
59         min_free_gb=float(tn.get("min_free_gb", 0.0)),
60     )

```

21. user (2026-06-19T16:46:57.900Z)

```

281:     def trajectory_to_action_windows(
290:         max_window_duration: float = 0.0,
291:         min_split_gap: float = 0.5,
292:         window_hard_cap: bool = False,
296:         window_blob_fallback: bool = False,
308:         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
313:         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing fix
from
317:         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when an
324:         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter): unlike
325:         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on every
gap-less
328:         ``max_window_duration`` once the gap-split and the principled rally-end trim have
both had
331:         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
333:         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until measured.
369:         if max_window_duration > 0:
371:             groups, int(max_window_duration * fps), int(min_split_gap * fps),
372:             hard_cap=window_hard_cap)
389:         if window_blob_fallback and max_window_duration > 0:
391:             groups, int(max_window_duration * fps), window_blob_factor)
397:             w["forced_cut"] = True
402:     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
439:     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames: int,
479:     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,

```

22. assistant (2026-06-19T16:47:00.192Z)

Now let me read the full `trajectory\_to\_action\_windows` method including the W1 split, R4 trim, and R5 blob fallback implementation.

23. user (2026-06-19T16:47:01.287Z)

```

281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,
294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:

```

```

299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309         the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310         and a single window swallows several rallies + the dead time between them. When
set, any
311         window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312         longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313         that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314         ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315         drop, and it is config-gated default-OFF until measured on the golden set.
316
317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
318         over-long window has NO internal inactivity gap to split on (a continuously-
active
319         trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320         blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322         (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341

```



```

342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]
367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
378         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379         if window_inactivity_end > 0:
380             tw = int(window_inactivity_end * fps)
381             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                         inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)
392
393         windows = []
394         for g, forced in zip(groups, forced_flags):
395             w = _finalize(g)
396             if forced:

```

```

397         w["forced_cut"] = True
398         windows.append(w)
399         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
404 float]]:
405         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
406         *slowly*
407         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
408         windowing
409         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
410         ``rally_thresh``
411         (higher than the active threshold). The rally is "on" at frame ``i`` while its
412         trailing
413         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
414         back to the
415         LAST such frame ? everything after is sustained slow drift (rally over). Only
416         cuts (recall
417         can't rise); a window whose tail never goes slow is untouched, and if no fast-
418         dense point
419         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
420         pointer; the START
421         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
422         if trail_frames <= 0 or len(group) < 2:
423             return group
424         frames = [f for f, _ in group]
425         fast = [1 if v > rally_thresh else 0 for _, v in group]
426         lo = 0
427         fast_sum = 0
428         last_dense = -1
429         for i in range(len(frames)):
430             fast_sum += fast[i]
431             while frames[lo] <= frames[i] - trail_frames:
432                 fast_sum -= fast[lo]
433                 lo += 1
434             win_count = i - lo + 1
435             if win_count > 0 and fast_sum / win_count >= min_density:
436                 last_dense = i
437         return group[: last_dense + 1] if last_dense >= 0 else group
438
439     @staticmethod
440     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
441         """Index of the active frame nearest the group's temporal midpoint (used to chop
442         a
443         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
444         fallback."""
445         mid = (group[0][0] + group[-1][0]) / 2.0
446         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
447         return split_i
448
449     @staticmethod
450     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
451 int,
452 min_split_frames: int,
453 hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
454         """Recursively split any active-frame group longer than ``max_win_frames`` at
455         its largest
456         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
457         likely rally
458         boundary. Splitting only subdivides existing groups (never merges/extends), so
459         it cannot
460         invent or drop coverage; a group with no gap ? the floor is left intact (a
461         genuinely long

```

```

446 rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
447 temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
448 bound on continuously-active trajectories). Iterative worklist to bound stack
depth.
449 if max_win_frames <= 0:
450     return groups
451 out: List[List[Tuple[int, float]]] = []
452 work = list(groups)
453 while work:
454     g = work.pop()
455     if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
456         out.append(g)
457         continue
458     best_i, best_gap = -1, -1
459     for i in range(1, len(g)):
460         gap = g[i][0] - g[i - 1][0]
461         if gap > best_gap:
462             best_gap, best_i = gap, i
463     if best_i < 0 or best_gap < min_split_frames:
464         # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
465         # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
466         if hard_cap:
467             split_i = TrackNetRunner._midpoint_split_index(g)
468             work.append(g[:split_i])
469             work.append(g[split_i:])
470         else:
471             out.append(g)
472             continue
473         work.append(g[:best_i])
474         work.append(g[best_i:])
475     out.sort(key=lambda grp: grp[0][0])
476     return out
477
478 @staticmethod
479 def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                        blob_factor: float = 4.0
481                        ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482     """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483 inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
484 ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485 signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486 11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
487 is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
488 midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489 degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490 rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491 can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492 that piece came from a forced chop. Iterative worklist to bound stack depth.
493 if max_win_frames <= 0:
494     return groups, [False] * len(groups)
495     blob_frames = max_win_frames * max(1.0, blob_factor)
496     out: List[List[Tuple[int, float]]] = []

```

```

497         flags: List[bool] = []
498         forced = 0
499         for g in groups:
500             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501                 out.append(g)
502                 flags.append(False)
503                 continue
504             pieces: List[List[Tuple[int, float]]] = []
505             work = [g]
506             while work:
507                 h = work.pop()
508                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                     pieces.append(h)
510                     continue
511                 split_i = TrackNetRunner._midpoint_split_index(h)
512                 work.append(h[:split_i])
513                 work.append(h[split_i:])
514             pieces.sort(key=lambda grp: grp[0][0])
515             out.extend(pieces)
516             flags.extend([True] * len(pieces))
517             forced += len(pieces) - 1
518         if forced:
519             logger.warning(
520                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "

```

24. assistant (2026-06-19T16:47:05.208Z)

I have the full W1/R4/R5 mechanics. Now let me verify the Gate A net-crossing logic in `rally\_gate.py` (min_crossings default, median play-axis estimate) and check whether the WASB runner subclasses this or has its own implementation.

25. user (2026-06-19T16:47:06.243Z)

```

1         """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3         WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4         the opponent for service after losing a point, that single trajectory looks like a
5         rally. The discriminator is structural, not duration-based: a real rally has the
6         shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7         single service feed crosses ~once.
8
9         This module is camera-agnostic and needs NO new model ? it works on the trajectory
10        we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11        axis with the larger shuttle spread) and the net position (median shuttle coord on
12        that axis ? the shuttle clusters at the two ends and crosses the net region often,
13        so the median sits near the net), then counts sign changes of (coord - net) across
14        the visible points inside a candidate window, with a deadband to ignore jitter.
15
16        Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17        Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18        windows from trajectory_to_action_windows, or to fold into it later.
19        """
20        from __future__ import annotations
21
22        from dataclasses import dataclass
23        from typing import List, Optional, Sequence, Tuple
24
25        Interval = Tuple[float, float]
26
27
28        @dataclass
29        class GatedWindow:
30            start: float
31            end: float

```

```

32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play

```

```

96         axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98         `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99         `(axis, net, span)` (from `estimate_play_axis`) so the  $O(P \log P)$  estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                   kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

26. assistant (2026-06-19T16:47:13.436Z)

Now I have the full picture on Gate A. Key finding: the function-signature default ``min_crossings=2`` in ``rally_gate.py``, but the `**served/config` default is ``min_crossings=0`` (OFF) in ``FusionConfig``. The net position is the `**median**` shuttle coord on the play axis ? matches the review's "median play-axis" claim. Let me verify the WASB runner uses the same ``trajectory_to_action_windows`` (it's on ``TrackNetRunner`` ? is ``WasbRunner`` a subclass?) and confirm the Gemini provider is the handoff target. Let me also check the CODE_MAP for the documented pipeline shape, and the padding location in stitching vs handoff (the review says "Phase 3 padding ~2s each end to a Gemini provider").

27. user (2026-06-19T16:47:15.048Z)

```

20:# parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on TrackNetRunner.
21:from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
68:class WasbRunner(WslCommandMixin, DetectorRunner):
198:     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199:     trajectory_to_action_windows = staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

28. user (2026-06-19T16:47:17.072Z)

```

87:-> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
123:[Omitted long matching line]
144:[Omitted long matching line]
156:[Omitted long matching line]
157:[Omitted long matching line]
161:[Omitted long matching line]
167:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
168:- `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing + net-
crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn` (adds
`rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a labels
file yields no rallies), `::run`/`::main`. $

```

```
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
175:[Omitted long matching line]
181:[Omitted long matching line]
232:$ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
312:| Tune candidate windowing (velocity/merge/min-dur) |
`@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
349:| `test_fusion_hybrid.py` | `pipeline/segmenters/fusion_hybrid.FusionSegmenter`
(net_crossings persisted; superset; Gate A & Gate B handoff gating; person-cue features;
`_build_runner` substrate select) |
352:| `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`,
`from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck,
run_predict, stage) |
353:| `test_wasb_runner.py` | `pipeline/detectors/wasb_runner.WasbRunner` WSL-glue
(DetectorRunner conformance, healthcheck, `parse_trajectory_csv`/`trajectory_to_action_windows`
rebind, run_predict; WSL mocked) |
```

29. assistant (2026-06-19T16:47:22.531Z)

Confirmed: `WasbRunner` rebinds `trajectory\_to\_action\_windows` from `TrackNetRunner` (the shared windowing brain). The narrative's pipeline maps to real code. Now let me verify the Gemini provider handoff (the review says "to a Gemini provider") and confirm the `\_active\_frames` velocity normalization. Let me also double-check whether the review's "padding ~2s each end" is correct (config default `ai\_handoff\_padding: 2.0`, applied in Phase 3 ? confirmed in trajectory_hybrid lines 278-279). Let me verify the default provider and the velocity normalization detail.

30. user (2026-06-19T16:47:24.012Z)

```
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252             points.sort(key=lambda p: p.frame)
253             return points
254
255     @staticmethod
256     def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                       velocity_thresh: float) -> List[Tuple[int, float]]:
258         """Per-frame `(frame_idx, velocity)` for frames where the shuttle is in
flight.
259
260         Computes each visible point's normalised velocity from the last *visible* point;
261         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263         the caller so identical floats feed the velocity math."""
264         active = [] # (frame_idx, velocity)
265         prev: Optional[TrajectoryPoint] = None
266         for p in points:
267             if not p.visible:
268                 prev = None # break the trajectory; absent shuttle = not in flight
269                 continue
```

```

270         if prev is not None:
271             dframes = p.frame - prev.frame
272             if dframes > 0:
273                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                 velocity = (dist / frame_width) * (fps / dframes)
275                 if velocity > velocity_thresh:
276                     active.append((p.frame, velocity))
277             prev = p
278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(

```

31. assistant (2026-06-19T16:47:29.360Z)

All mechanics confirmed. Now let me verify the default provider/segmenter wiring. The narrative says the handoff goes "to a Gemini provider." The config `default\_segmenter` is `yolo\_hybrid` (not a trajectory hybrid), and `ai\_provider: "gemini"`. Let me quickly check the Gemini provider exists and is the default, and confirm there's no `RALLY\_DETECTION\_CODE\_REVIEW.md` anywhere via a final broad git check.

32. user (2026-06-19T16:47:32.068Z)

```

===provider registry default===
backend/providers/__init__.py:3:from backend.providers.gemini import GeminiProvider
backend/providers/__init__.py:9:         self.register("gemini", GeminiProvider)
backend/providers/gemini.py:19:class GeminiProvider(AIVideoProvider):
===any tracked/untracked review file anywhere on disk (non-worktree)===
DONE

```

33. assistant (2026-06-19T16:47:46.448Z)

Confirmed: the `RALLY\_DETECTION\_CODE\_REVIEW.md` does not exist in this repo (tracked, untracked, in any branch, worktree, or stash). The Gemini provider is registered as the default `ai\_provider`.

I now have everything needed to verify the narrative's claims. Let me compile the structured assessment.

Summary of my verification against the actual code:

```

**Config/default checks (all in `backend/config/models.py` unless noted):**
- `velocity_thresh ~0.02` ? CORRECT. `WasbIndexConfig.velocity_threshold = 0.02` (line 69),
  `TrackNetConfig = 0.02` (line 43), read at `trajectory_hybrid.py:160` defaulting to 0.02.
- `merge_gap ~2.0s` ? CORRECT. `dead_time_merge_gap = 2.0` (line 122).
- `max_window_duration` ? review implies W1 with split; **actual default is 6.0 (default-ON),
  not 0/OFF**. The function signature default is 0.0 but the config ships 6.0 (R1 milestone, line
  139).
- `min_split_gap ~0.5` ? CORRECT. `window_min_split_gap = 0.5` (line 140).
- `window_hard_cap` chop-in-half ? CORRECT mechanism, **default False/OFF** (line 145).
- `padding ~2s each end` ? CORRECT. `ai_handoff_padding = 2.0` (line 172), applied Phase 3 at
  `trajectory_hybrid.py:278-279`.
- `min_crossings ~2` ? **MIXED**. The `rally_gate.py` function-signature default is 2 (lines 92,
  114), but the **served config default is `min_crossings = 0` (OFF)** in `FusionConfig` (line
  105). Gate A does not fire by default.
- `rally_thresh/min_density` (R4) ? defaults `inactivity_rally_thresh = 0.20`,
  `inactivity_min_density = 0.30`, `window_inactivity_end = 1.5` (default-ON, R1 milestone). Note:
  function-signature defaults differ (0.08 / 0.34 / 0.0).

```

```

**Label/flag checks:** W1 (split), hard-cap, R4 (`_trim_inactive_tail`), R5
(`_blob_fallback_chop` with `forced_cut=True` flag), Gate A (`rally_gate.gate_windows`, net =
median play-axis coord) ? all real and faithfully described.

```

```

**vs `docs/HOW_RALLY_DETECTION_WORKS.md`:** that doc only describes the 3-knob model

```


(velocity_thresh / merge_gap / min_window_duration) plus "where sparks could land." It does NOT document W1/R4/R5/Gate A as part of the live pipeline ? those live in `RALLY\_QUALITY\_RESEARCH.md` / `QUALITY\_ITERATIONS.md` and the code. So the review's richer pipeline is real in code but goes beyond this particular doc's mental model.

34. user (2026-06-19T16:48:27.325Z)

Structured output provided successfully

35. assistant (2026-06-19T16:48:33.533Z)

I've completed the verification and submitted the structured output.

Key findings: The `RALLY\_DETECTION\_CODE\_REVIEW.md` file does not exist anywhere in the repo, so I verified the narrative's enumerated claims against the actual code rather than the review prose. The pipeline mechanics and flag names (W1 split, hard-cap, R4 trim, R5 blob with `forced\_cut`, Gate A net-crossing on median play-axis, padding, Gemini handoff) are all real and faithfully described in `backend/pipeline/detectors/tracknet\_runner.py`, `fusion\_hybrid.py`, and `rally\_gate.py`. The two main accuracy traps are (1) several cited "defaults" are function-signature defaults that disagree with the shipped config defaults (`max\_window\_duration` 0?6.0, R4 thresholds, `min\_crossings` 2?0/OFF), and (2) `docs/HOW\_RALLY\_DETECTION\_WORKS.md` only documents the 3-knob model and lists Gate A/fusion as a not-yet-built open question, so the richer pipeline matches the code but not that doc.